Recap

HTTP/2 introduces new features mainly related to the usage of network resources

Multiplexing of streams and frames over a single TCP connection (framing layer is responsible of reassembling the frames)

Flow control regulates the rate of data transmission between clients and servers

Priorities associated with streams specify how to allocate the bandwidth and computation resources across streams

The settings associated with flow control and prioritization are very critical

Headers are compressed using the HPACK method to reduce the overhead associated with HTTP requests and responses

Static and dynamic tables updates

```
ET /lite/static/js/7979.5e7e834d.chunk.js HTTP/2
ost: cdn-client.medium.com
ser-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:150.0) Gecko/20100101 Firefox/150.0
ccept: */*
ccept-Language: en-US,en;q=0.9
ccept-Encoding: gzip, deflate, br, zstd
eferer: https://medium.com/coderscorner/http-2-flow-control-77e54f7fd518
onnection: keep-alive
ookie: uid=lo_4b9d2fe3066f; sid=1:EIbLeNwCI5r2k0GVL270twOS3awWYmHUxjkO4yPpI7VkxO63ZQeCBiD8q9dtSp5x;
ec-Fetch-Dest: script
ec-Fetch-Mode: no-cors
ec-Fetch-Site: same-site
ragma: no-cache
```

Index	Header Name	Header Value
1	:authority	
2	:method	GET
3	:method	POST
4	:path	/
5	:path	/index.html
6	:scheme	http
7	:scheme	https
8	:status	200
9	:status	204
10	:status	206
11	:status	304
12	:status	400
13	:status	404
14	:status	500
15	accept-charset	
16	accept-encoding	gzip, deflate
17	accept-language	
18	accept-ranges	
19	accept	
20	access-control-allow-origin	
21	age	
22	allow	
23	authorization	
24	cache-control	
25	content-disposition	
26	content-encoding	
27	content-language	
28	content-length	
29	content-location	
30	content-range	
31	content-type	
32	cookie	
33	date	
34	etag	
35	expect	
36	expires	
37	from	
38	host	
39	if-match	
40	if-modified-since	
41	if-none-match	
42	if-range	
43	if-unmodified-since	
44	last-modified	
45	link	
46	location	
47	max-forwards	
48	proxy-authenticate	
49	proxy-authorization	
50	range	
51	referer	
52	refresh	
53	retry-after	
54	server	
55	set-cookie	
56	strict-transport-security	
57	transfer-encoding	
58	user-agent	
59	vary	
60	via	
61	www-authenticate	

New updates

```
GET /lite/static/js/7975.3f8d607c.chunk.js HTTP/2
Host: cdn-client.medium.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:150.0) Gecko/20100101 Firefox/150.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br, zstd
Referer: https://medium.com/coderscorner/http-2-flow-control-77e54f7fd518
Connection: keep-alive
Cookie: uid=lo_4b9d2fe3066f; sid=1:EIbLeNwCI5r2k0GVL270twOS3awWYmHUxjkO4yPpI7VkxO63ZQeCBiD8q9dtSp5x;
Sec-Fetch-Dest: script
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: same-site
Pragma: no-cache
```


HPACK performance

Main benefits of header compression:

- Reduction of bandwidth usage
- Increase content delivery speed

On average compression reduces the size of headers by over 30%

In general request headers get better compression

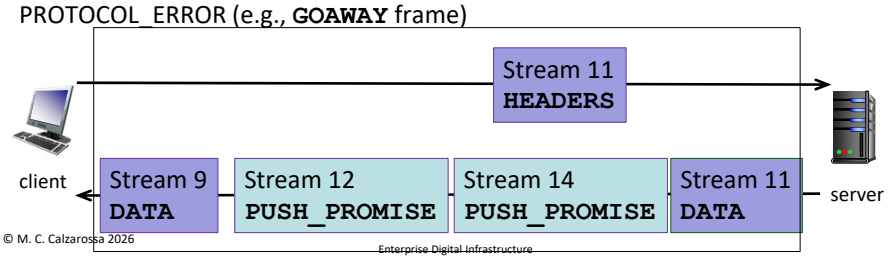
Server push

Server push is a feature that allows servers to send responses to a client in association with a previous client-initiated request with the objective of improving client-perceived performance

Servers "predict" what requests will follow those that they have received

Clients should know which objects servers intend to push (duplicated requests)

Servers sends a **`PUSH_PROMISE`** frame which contains the HTTP headers of the promised object (inside a server-initiated stream); clients might refuse objects by resetting the stream (e.g., **`RST_STREAM`** frame) or by sending a PROTOCOL_ERROR (e.g., **`GOAWAY`** frame)



Implementation

In principle server push is a good feature, but its implementation is very challenging

Servers need to correctly anticipate the additional requests a client will make, by taking into account factors such as caching, client resources, user behavior

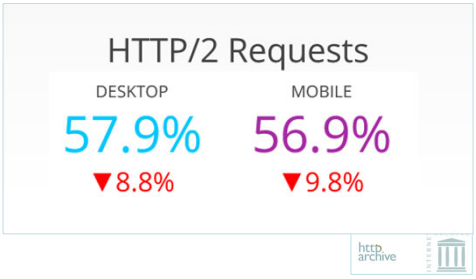
Errors in predictions can lead to performance degradation: pushing significant amount of data can cause contentions with responses of critical resources

Very limited adoption

This feature can be implemented using "Early Hints", i.e., the server sends an informational response with status code 103 together with a **`link`** response header with the **`preload`** or **`preconnect`** keywords and the client will send the request ahead of the parsing

```
103 Early Hints
link: </scripts/jquery.js>; rel=preload
```

HTTP/2 adoption



Variations refer to the period 2020-2026

HTTP/3

The Hypertext Transfer Protocol Version 3 (HTTP/3) is the youngest member of the HTTP protocol family [RFC 9114 "HTTP/3" published in 2022]

Same semantics over a **new transport protocol**, i.e., QUIC

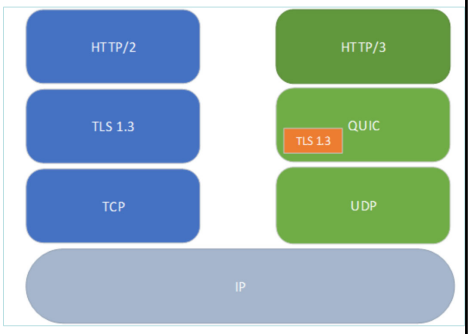
Five main ingredients:

1. *HTTP paradigms and concepts*
2. *HTTP/2 syntax*
3. QUIC
4. UDP
5. QPACK for compression (instead of HPACK)

QUIC

Quick UDP Internet Connection (QUIC) is a transport protocol built on top of UDP

Originally developed by Google (and deployed since 2014), QUIC aims at improving web communications and reducing latency [RFC 9000 "QUIC: A UDP-Based Multiplexed and Secure Transport" published in 2021]



QUIC implements a reliable transport at the application layer (running in the user space) by recreating TCP services

It manages stream multiplexing, flow control, congestion control, loss recovery, …

It implements encryption by including TLS 1.3 [RFC 9001 "Using TLS to Secure QUIC" published in 2021]

125

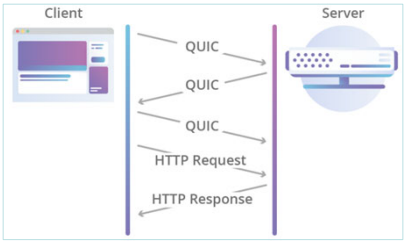
QUIC Initial Connection

Client sends a random 64-bit Connection ID (used to fight IP spoofing)

Server replies with a certificate and a cookie

Client responds with:

- CID
- cookie
- proposed encrypted session key
- encryption algorithm



Client can immediately send requests

Low latency connection establishment, i.e., 1-RTT for new exchanges, 0-RTT for clients and servers that have previously communicated

126

HTTP/3 features

QUIC connections are persistent across multiple requests

For best performance, clients will not close QUIC connections until no further communication with a server is necessary (e.g., when a user navigates away from a particular web page) or until the server closes the connection

All client-initiated bidirectional streams are used for HTTP requests/responses

HTTP/3 provides an internal framing layer similar to HTTP/2

Multiplexing is implemented using the QUIC stream abstraction

Header compression is based on QPACK, a variation of HTTP/2 HPACK that takes into account out-of-order delivery

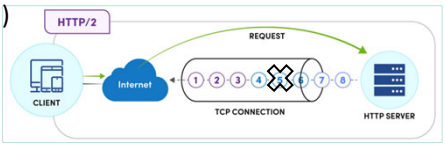
HTTP/3 aims at reducing delays especially on unstable connections like mobile data or public Wi-Fi

127

QUIC benefits for HTTP/3

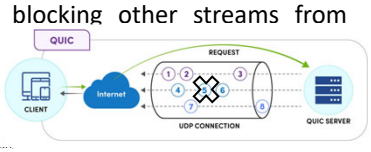
HTTP/3 overcomes HTTP/2 limitations due to TCP/TLS inefficiencies by improving how quickly connections recover when network conditions change

HTTP/2 stream multiplexing significantly improves performance, but it is not visible to TCP loss recovery mechanisms (e.g., lost/reordered packets cause all active streams to experience a stall)



HTTP/3 supports multiple independent streams: each HTTP stream is mapped to a different QUIC stream

When a packet is lost, only the affected stream will need to wait for the retransmission of the lost frames without blocking other streams from moving forward



128

HTTP/3 (cont'd)

An HTTP/3 connection is only set up after some initial resources have been downloaded over an HTTP/1.1 or HTTP/2 connection

Browsers assume by default that servers do not support HTTP/3 and send the first request via either HTTP/2 or HTTP/1.1 on a TCP/TLS connection

If the server supports HTTP/3, it responds with an **`alt-svc`** header field

The browser will open a QUIC connection or wait until TCP connection is closed

```
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```


HTTP/3 request message

```
GET /pdata/ot/202501.2.0/prod/scripttemplates/otSDKStub.js HTTP/3
Host: www.akamai.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br, zstd
Referer: https://www.akamai.com/it
DNT: 1
Alt-Used: www.akamai.com
Connection: keep-alive
Cookie: AKA_A2=A; _abck=3EFDDC044DD99AF9CC4E0ABB01B0B3C7~-1~YAAQvRRlX8ol+UCWAQAAMzHIeA0HTXAxDq
Sec-Fetch-Dest: script
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: same-origin
Priority: u=2
Pragma: no-cache
Cache-Control: no-cache
```

HTTP/3 response message

```
HTTP/3 200
content-type: application/x-javascript
accept-ranges: bytes
content-encoding: br
etag: "160781b098f2515908d071936ad73582:1741151847.396579"
last-modified: Wed, 23 Apr 2025 09:44:42 GMT
content-length: 6783
cache-control: max-age=4147
date: Sun, 27 Apr 2025 19:45:08 GMT
server-timing: cdn-cache; desc=HIT
server-timing: edge; dur=39
quic-version: 0x00000001
alt-svc: h3=":443"; ma=93600
strict-transport-security: max-age=31536000 ; includeSubDomains ; preload
expect-ct: max-age=3600, report-uri=https://reporting.go-mpulse.net/report/FDSGP-LEB9B-T8Y2A-5V5ED-
nel: {"report_to":"default","max_age":3600,"include_subdomains":true}
content-security-policy-report-only: report-uri https://reporting.go-mpulse.net/report/FDSGP-LEB9B-
report-to: {"max_age":3600,"endpoints":[{"url":"https://reporting.go-mpulse.net/report/FDSGP-LEB9B-
x-content-type-options: nosniff
x-xss-protection: 1; mode=block; report=https://reporting.go-mpulse.net/report/FDSGP-LEB9B-T8Y2A-5V
x-frame-options: SAMEORIGIN
akamai-grn: 0.ae14655f.1745783108.22f00fe8
set-cookie: _abck=3EFDDC044DD99AF9CC4E0ABB01B0B3C7~-1~YAAQrhRlX2OTlUOWAQAAjTPIeA3acl2akzEqZ176zDh+
```


HTTP/3 adoption

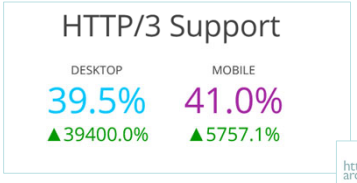
HTTP/3 offers user-centric benefits in terms of Page Load Time

The benefits for software agents designed to asynchronously crawl and index content are minimal

HTTP/3 supported by

- Browsers
- CDN providers
- Some web servers (e.g., nginx, caddy)

Big players (e.g., Google) are moving towards HTTP/3



Variations refer to 2020-2026 period